[AI Native Camp] Day 4 — Context Engineering + Compound Engineering: 만든 걸 복리로 키우는 법

목차

> 1. AI와 함께 일하는 나만의 운영체제(OS), 클로드 코드
> 2. AI를 똑똑하게 만드는 비법: 맥락 엔지니어링 (Context Engineering)
> 3. AI와 함께 일하는 시스템의 안전망, 깃허브 (GitHub)
> 4. AI를 복리처럼 똑똑하게 만드는 컴파운드 엔지니어링 (Compound Engineering)
> 5. 컴파운드 엔지니어링을 위한 '랩 스킬'과 폴더 구조

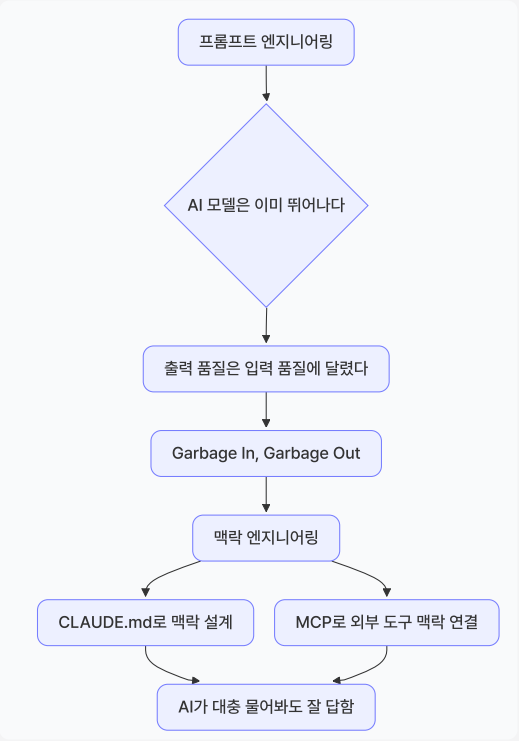
AI를 똑똑하게 만들어서 내 일을 더 잘하게 하는 방법이 궁금하지 않아? 우리가 만든 AI 도구들을 그냥 한 번 쓰고 마는 게 아니라, 계속 고치고 발전시켜서 마치 복리처럼 효과를 키우는 방법이 있어. 오늘은 AI와 함께 일하는 시스템을 만들고, 그걸 계속 유지보수하면서 더 똑똑하게 만드는 '컴파운드 엔지니어링'에 대해 알아볼 거야.

1. AI와 함께 일하는 나만의 운영체제(OS), 클로드 코드

Claude Code: 운영체제(OS) 메타포

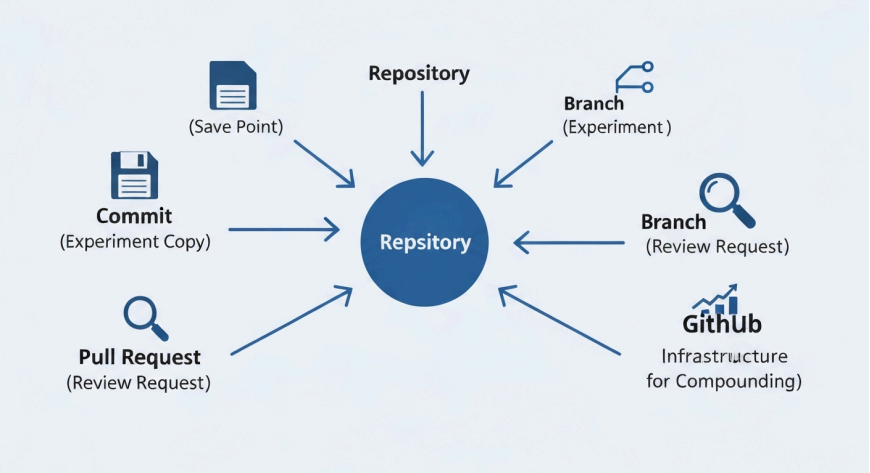
- 우리가 컴퓨터를 쓸 때 운영체제(OS)가 하드웨어를 제어하고, 앱을 실행하고, 파일을 관리해주잖아? 클로드 코드(Claude Code)도 비슷하게 우리가 일할 때 쓰는 도구들을 제어하고, 작업 흐름(워크플로우)을 실행하고, 필요한 정보(맥락)를 관리해주는 역할을 해.
- 예전에는 챗GPT 같은 '대화창'에 질문하고 답을 받았다면, 클로드 코드는 '터미널'이라는 컴퓨터 환경에서 직접 일하면서 우리가 쓰는 도구들을 연결하고, 스킬을 만들어서 작업을 자동화하고, 파일 시스템으로 맥락을 관리할 수 있게 해줘.
- 이건 마치 "이거 해줘" 한마디면 AI가 내가 설정한 맥락(CLAUDE.md)을 이해하고, 필요한 데이터(MCP 연결)를 읽어서, 정해진 작업 흐름(스킬)을 실행하는 것과 같아.

2. AI를 똑똑하게 만드는 비법: 맥락 엔지니어링 (Context Engineering)



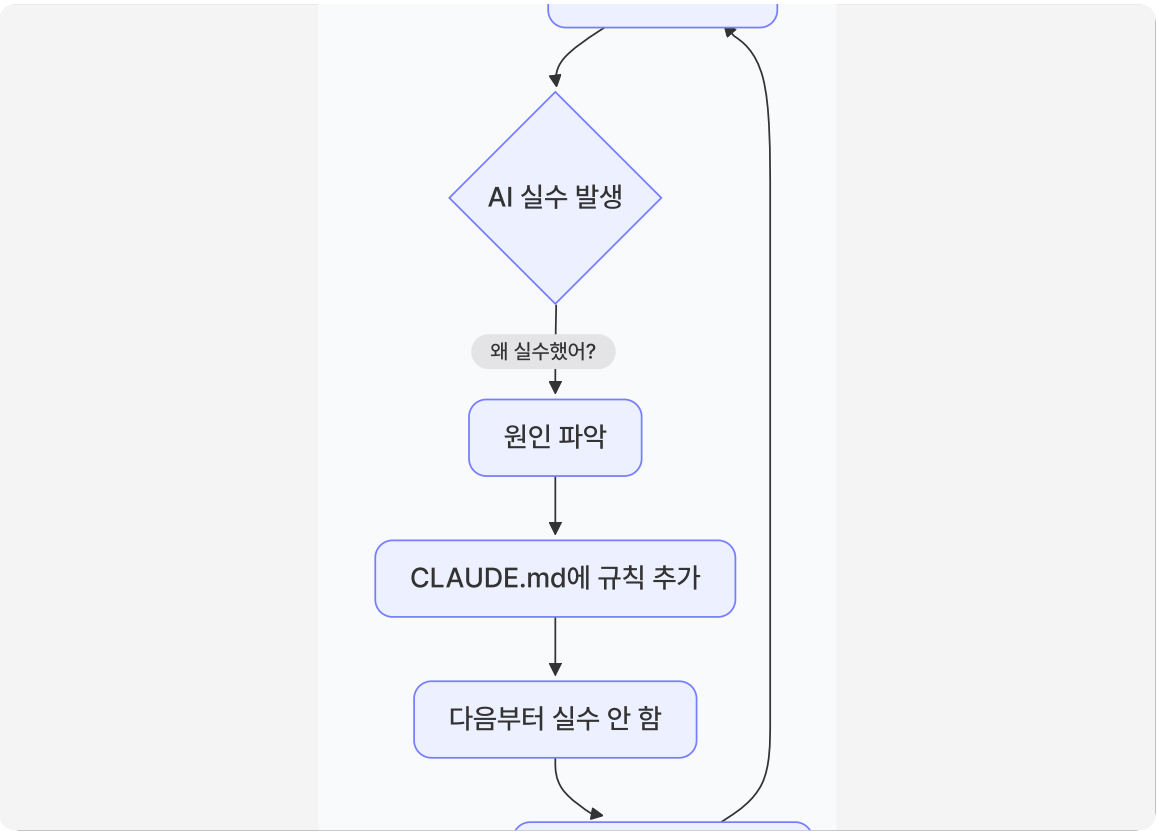
- 예전에는 AI에게 질문(프롬프트)을 잘 쓰는 게 중요하다고 생각했어. AI 모델은 이미 똑똑하니까, 좋은 질문을 해야 좋은 답을 얻을 수 있다고 본 거지.
- 하지만 2025년쯤부터는 '맥락 엔지니어링'이라는 개념이 중요해지기 시작했어. 이건 단순히 질문을 잘하는 걸 넘어서, AI가 참고할 수 있는 전체적인 정보(맥락)를 설계하고 관리하는 걸 말해.
- 우리가 이미 캠프 첫날부터 CLAUDE.md를 만들어서 내 맥락을 설계하고, MCP로 외부 도구의 맥락을 연결한 것도 다 이 맥락 엔지니어링의 과정이었어.
- 맥락 엔지니어링은 AI가 대충 물어봐도 잘 답하게 만들어줘. 한 번 잘 설계해두면 매번 좋은 결과를 얻을 수 있는 거지.

3. AI와 함께 일하는 시스템의 안전망, 깃허브 (GitHub)



- 우리가 AI와 함께 만든 CLAUDE.md, 스킬, MCP 설정 같은 것들은 한 줄만 바뀌어도 결과가 달라질 수 있어. 그래서 이런 변경 사항들을 안전하게 관리하는 시스템이 필요해.
- 깃허브(GitHub)는 이런 AI 작업물들을 저장하고, 공유하고, 변경 이력을 남기고, 여러 사람이 함께 작업(협업 리뷰)할 수 있게 해주는 곳이야. 마치 게임의 '세이브 포인트'처럼 잘 되는 시점에 저장하고, 왜 바꿨는지 메모를 남기는 걸 '커밋(Commit)'이라고 해.
- **레포지토리(Repository)**: 내 프로젝트의 집이라고 보면 돼. CLAUDE.md, 스킬 파일 같은 것들이 모여 있는 곳이야.
- **브랜치(Branch)**: 원본을 건드리지 않고 새로운 시도를 할 수 있는 복사본이야. 마치 PPT를 여러 명이 나눠서 만들 때 각자 자기 부분을 작업하는 것과 같아.
- **풀 리퀘스트(Pull Request, PR)**: 내가 바꾼 내용을 원본에 합치기 전에 다른 사람에게 검토를 요청하는 거야. 마치 결재 문서를 올리는 것과 비슷해.
- **푸시(Push)**: 내 컴퓨터(로컬)에서 저장한 변경 사항(커밋)들을 깃허브(원격 저장소)에 올리는 행위를 말해.
- 깃허브는 우리가 만든 것을 고칠 때마다 변경 이력을 자동으로 쌓아줘. 그래서 만약 문제가 생기면 과거 버전으로 쉽게 되돌릴 수 있지.

4. AI를 복리처럼 똑똑하게 만드는 컴파운드 엔지니어링 (Compound Engineering)



- 컴파운드 엔지니어링은 AI에게 일을 시킬 때마다 AI를 더 똑똑하게 만들어서, 업무 효율을 마치 복리 이자처럼 계속 불려나가는 방법이야.
- 이건 단순히 AI를 한 번 잘 만들어서 끝내는 게 아니라, 계속 사용하면서 불편한 점을 찾고, 고치고, 기록하는 과정을 반복하는 거야.
- AI가 실수를 하면 "왜 실수했어?" 하고 원인을 파악한 다음, CLAUDE.md 같은 곳에 새로운 규칙을 추가해서 다음부터는 같은 실수를 하지 않도록 만드는 거지.
- 이런 과정을 계속 반복하면 AI는 점점 더 똑똑해지고, 나만의 완벽한 파트너가 될 수 있어. 마치 '실수 방지 노트'를 만들어서 AI를 훈련시키는 것과 같아.
- 실제로 AI 모델 자체를 업그레이드하는 것보다, 이런 '에이전틱 워크플로우'를 잘 활용하는 것이 AI의 정확도를 훨씬 더 크게 높여준다는 연구 결과도 있어.

5. 컴파운드 엔지니어링을 위한 '랩 스킬'과 폴더 구조

- '세션 랩(Session Lab)' 스킬은 우리가 AI와 함께 작업한 세션이 끝났을 때, 마치 퇴근 전에 책상을 정리하는 것처럼 자동으로 작업 내용을 분석하고 정리해주는 도구야.
- 이 스킬은 여러 전문가(서브 에이전트)를 동시에 활용해서
  - 업데이트가 필요한 문서를 찾아내고 (문서 업데이트 전문가)
  - 자동화할 수 있는 작업을 제안하고 (자동화 스카우트)
  - 오늘 배운 내용을 정리하고 (학습 정리 전문가)
  - 다음에 할 일을 제안해줘 (팔로우업 제안 전문가)
- 이렇게 여러 전문가가 분석한 내용을 모아서 중복되는 부분을 걸러내고, 최종적으로 사용자에게 어떤 작업을 할지 물어봐. 예를 들어, 깃허브에 커밋할지, 문서를 업데이트할지 등을 선택할 수 있게 해주는 거지.
- 컴파운드 엔지니어링을 잘 활용하려면, 현종님의 사례처럼 폴더 구조를 잘 만드는 것도 중요해. 여러 프로젝트나 스킬들을 하나의 큰 프로젝트 폴더 안에 넣고, 클로드 코드를 그 폴더에서 실행하면 AI가 전체 맥락을 더 잘 이해하고 활용할 수 있게 돼.
- 이런 폴더 구조를 잘 만들고 '랩 스킬'을 꾸준히 사용하면, 우리가 만든 AI 시스템이 계속해서 발전하고, 업무 효율이 복리처럼 쌓이는 경험을 할 수 있을 거야.

글을 작성하세요. 블록을 추가하려면 '/'를 입력하세요